Department of Computer Engineering and Technology

Computer Networks Laboratory (CSE20110)

Laboratory Continuous Assessment 3: Mini-Project Report

Cross-Layer Delay-Gradient Adaptive Video/Audio Streaming Simulator with Dynamic XOR FEC and Audio-First Survival

Submitted by

Sr. No.	Name	PRN
1	Mahi Sanghani	1262242807
2	Lakshy Narang	1262242935
3	Aniket Chaudhari	1262243379
4	Soniya Jhadav	1262243503

Source code (GitHub): https://github.com/lakshynarang/cross-layer-abr-streaming-simulator

Group	Group No. 14, Panel K, Batch 2 and 3
Class	Third Year B.Tech. Computer Engineering and Technology
Faculty guide	Dr. Uma Pujeri, Dr. Swati
Academic year	2026–2027
Submission date	23 October 2026
Place	Pune

Certificate

This is to certify that the mini-project report titled “Cross-Layer Delay-Gradient Adaptive Video/Audio Streaming Simulator with Dynamic XOR FEC and Audio-First Survival” has been successfully completed by the following students of Third Year B.Tech. Computer Engineering and Technology, Group No. 14, Panel K, Batch 2 and 3:

Sr. No.	Name	PRN
1	Mahi Sanghani	1262242807
2	Lakshy Narang	1262242935
3	Aniket Chaudhari	1262243379
4	Soniya Jhadav	1262243503

The work was carried out under the guidance of our faculty guides Dr. Uma Pujeri and Dr. Swati as part of the Computer Networks Laboratory (CSE20110), Laboratory Continuous Assessment 3, during the academic year 2026–2027. To the best of our knowledge, the work is original and satisfies the requirements of the course.

_____

Dr. Uma Pujeri, Dr. Swati
Faculty Guide, Department of Computer Engineering and Technology

_____

Head of Department
Department of Computer Engineering and Technology

Date: 23 October 2026 Place: Pune

Acknowledgement

We express our sincere gratitude to our faculty guides, Dr. Uma Pujeri and Dr. Swati, for their steady guidance, technical feedback and encouragement throughout this mini-project. Their questions on queueing delay, loss classification and forward error correction shaped both the design and the evaluation of our work.

We also thank the Department of Computer Engineering and Technology for providing the academic environment and resources for this project, and the laboratory staff for their support with the computing facilities and networking tools used in our experiments.

Finally, we thank our families and classmates for their support and constructive suggestions during the project.

Abstract

Conventional Adaptive Bitrate (ABR) streaming protocols such as MPEG-DASH and Apple HLS operate strictly at the application layer over HTTP/TCP, suffering from an inability to differentiate between non-congestive wireless channel fades and physical router bufferbloat. Consequently, transient wireless interference triggers catastrophic video downshifts to 240p and severe rebuffering stalls. This project designs and implements a cross-layer transport-aware multimedia streaming simulator operating over raw Berkeley UDP sockets with a custom 16-byte RTP-like binary header. By continuously tracking minimum propagation delay and instantaneous round-trip time, a Dual-Metric Classifier isolates bottleneck router queueing delay ($D_{queue} = RTT - RTT_{min}$). Under non-congestive wireless packet loss, the system maintains 720p HD streaming and activates dynamic 20% Galois-Field XOR Forward Error Correction (RFC 5109) with 2D matrix burst-stride interleaving, achieving 100% zero-RTT recovery of multi-packet burst drops in unit testing. Under bottleneck bufferbloat ($D_{queue} > 25$ ms), the controller proactively steps down transmission bitrate before tail-drop packet loss occurs. Under catastrophic link degradation (>20% loss), Audio-First Graceful Degradation dedicates link capacity to 3x redundant audio packets. Quantitative evaluation across a 60-second multi-regime trace demonstrates 100% elimination of rebuffering stalls (0.00s vs 16.70s in BBA), a 51.6% packet recovery efficiency using sequential block XOR parity under sustained Markov burst fading, 97.5% audio survival, and a 103.6% improvement in Pensieve QoE score (+128.40 vs +63.05).

Keywords: Adaptive Bitrate Streaming, Cross-Layer Optimization, Forward Error Correction, Queueing Delay Gradient, UDP Socket Programming, Multimedia Quality of Experience (QoE).

Table of Contents

Certificate **2**

Acknowledgement **3**

Abstract **4**

1 Introduction and Project Objectives **9**

1.1 Introduction and Background 9

1.2 Problem Statement 9

1.3 Scope 10

1.4 Objectives 10

2 Proposed Design and Requirements **11**

2.1 Architecture and Network Topology 11

2.2 Tools and System Requirements 12

2.3 Addressing and Protocol Selection 12

3 Implementation and Technical Working **13**

3.1 Methodology and Workflow 13

3.2 Configuration and Code Implementation 16

3.3 Protocols and Technical Working 17

3.4 Execution Procedure 18

4 Testing and Results **19**

4.1 Test Plan 19

4.2 Results and Execution Evidence 20

4.3 Objective Achievement Mapping 24

5 Discussion and Conclusion **24**

5.1 Discussion and Comparative Analysis 24

5.2 Advantages and Real-World Applications 25

5.3 Limitations 25

5.4 Challenges and Troubleshooting 26

5.5 Future Scope 26

5.6 Conclusion 26

References **27**

AI Usage Declaration **27**

Appendix A: Source Code Excerpts and Configuration **28**

A.1 protocol.py: header and flag packing 28

A.2 fec_engine.py: zero-padded XOR 28

A.3 fec_engine.py: 2D matrix interleaved encoding (Algorithm 2) 28

A.4 fec_engine.py: loss recovery with size restoration (Algorithm 3) 28

A.5 abr_controller.py: dual-metric classification (Algorithm 1) 29

A.6 network_channel.py: channel model parameters 29
Appendix B: Raw Telemetry and Wireshark Packet Evidence 29
B.1 Wireshark hex dumps (decoded with wireshark_dissector.lua) 29
B.2 Benchmark trace regimes (60 s) 30
Appendix C: Individual Student Contribution Statements 31

List of Figures

Figure 2.1: System architecture and dumbbell topology. Media packets travel from the server through the channel emulator to the client, and the client returns feedback directly to the server. 11

Figure 3.1: Workflow of one media chunk, from packetisation and parity generation through interleaving, the channel, loss classification and reconstruction, to playout and the feedback that drives the next chunk. 14

Figure 3.2: Implemented system running. The web dashboard (python web_dashboard.py, http://127.0.0.1:8080) at 54 s of the 60 s trace, with the channel sliders, the Dual-Metric Loss Classifier panel, the algorithm comparison table and Graphs 1 and 2. 18

Figure 4.1: Terminal output of python test_simulator.py -v showing tests T1 to T5 passing. 20

Figure 4.2: Four-panel benchmark output (benchmark_comparison.png): bitrate against link capacity, queueing delay with dynamic FEC share, client buffer occupancy, and the Pensieve QoE breakdown for the proposed controller, BBA and rate-based DASH. 21

Figure 4.3: Graph 3 shows the playback buffer of the proposed controller never reaching zero (zero stalls), and Graph 4 compares the net Pensieve QoE and stall penalties of the three controllers. 23

Figure 4.4: Header and XOR lab. Left: the 16-byte header inspector shows each serialised byte. Right: media packet P_1 is dropped and rebuilt as P_parity XOR P_0 XOR P_2 XOR P_3, giving an exact match in 0 RTT. 23

Figure 4.5: Interleaver proof for a burst of three consecutive wire packets. Standard 1D XOR recovers 0 of 3, while the stride-3 matrix interleaver spreads the burst over three groups and recovers 3 of 3. 24

List of Tables

Table 1.1: Project objectives and verification methods 10
Table 2.1: Tools and system requirements 12
Table 2.2: Hosts, ports and protocols 12
Table 2.3: Bitrate tiers 13
Table 3.1: Decision regimes of the Dual-Metric Classifier 14
Table 3.2: 16-byte transport header (network byte order) 17
Table 3.3: Flag bitfield (bytes 12–15) 17
Table 4.1: Test cases 19
Table 4.2: Phases of the 60-second benchmark trace 20
Table 4.3: Performance comparison on the 60-second multi-regime trace 21
Table 4.4: Objective achievement (Achieved / Partially achieved / Not achieved) 24
Table 5.1: Challenges encountered and how they were solved 26
Table A.1: Channel model parameters (network_channel.py) 29
Table B.1: Decoded fields of the audio packet 30
Table B.2: Decoded fields of the video packet 30
Table B.3: Benchmark trace regimes and the expected controller response 30
Table C.1: Individual contributions of the team members 31

1 Introduction and Project Objectives

1.1 Introduction and Background

Video is the dominant consumer of Internet bandwidth, widely reported at over 70% of traffic, and most of it is delivered by Adaptive Bitrate (ABR) streaming. In ABR, the server stores each video at several bitrates and the client picks a bitrate per chunk, trading picture quality against the risk of a playback stall. MPEG-DASH (ISO/IEC 23009-1) and Apple HLS (RFC 8216) standardise this model over HTTP and TCP.

The research literature has produced two main families of ABR controllers. Buffer-based adaptation (BBA) chooses the bitrate from the playback buffer level alone [2]. Learning-based controllers such as Pensieve train a neural policy against a quality-of-experience (QoE) reward [1]. Both still sit at the application layer and see the network only through chunk download times.

Transport research has meanwhile shown that queueing delay is a far better congestion signal than loss. TCP Veno uses the gap between expected and actual rate to decide whether a loss was random (wireless) or congestive [3]. Google's BBR estimates the minimum round-trip time and bottleneck bandwidth and keeps the queue short [4]. This project brings that cross-layer idea into ABR video, and pairs it with forward error correction so that wireless losses can be repaired without retransmission.

Real-time media is usually carried over UDP with an RTP-style header and RTCP-style feedback, as specified in RFC 3550 [5]. TCP, in contrast, enforces strict in-order byte delivery: one lost segment blocks every later frame (head-of-line blocking) until it is retransmitted. For live video and voice this converts a single wireless drop into a visible stall. XOR parity protection for RTP streams is defined in RFC 5109 [6], and the general principles of congestion control, queueing and multimedia networking are covered in [7].

1.2 Problem Statement

Conventional ABR controllers measure chunk download speed and playback-buffer occupancy. They have no view of the bottleneck router queue, so they cannot tell apart two very different situations:

- **True congestion:** the router FIFO queue is filling and queueing delay exceeds 25 ms.

- **Wireless channel fade:** packets are lost through multipath, interference or Wi-Fi contention while the queue is empty (queueing delay near 0 ms).

Because both look like slow chunks, the controller reacts to both as congestion. A transient 100 ms wireless fade makes TCP throttle itself, and the ABR controller "panic downshifts" from 1080p to 240p even though the link capacity is unchanged. TCP head-of-line blocking then drains the client buffer and causes rebuffering stalls. Fixed-rate FEC, the usual alternative, spends a constant overhead (for example 20%) on clear channels and still fails against correlated burst losses.

The problem addressed by this project is therefore: **how can a streaming system identify the true cause of loss, repair wireless loss locally with minimum overhead, slow down only when the queue really fills, and keep voice alive when the link collapses?**

1.3 Scope

The project covers interactive, low-latency video and audio streaming over raw UDP sockets. It includes a token-bucket bottleneck with a FIFO queue, jitter and Gilbert-Elliott Markov burst loss emulated by a proxy process, and a comparative benchmark of the proposed controller against a buffer-based (BBA) controller and a rate-based DASH controller. A web dashboard, Wireshark packet traces with a custom dissector, and a Mininet dumbbell topology support demonstration and verification. Live radio hardware (Wi-Fi or 5G base stations) is outside the scope.

1.4 Objectives

The objectives of the project, with the way each one is verified, are listed in Table 1.1.

Table 1.1: Project objectives and verification methods

Objective ID	Objective	Verification Method
O1	Design a compact 16-byte binary UDP transport header (struct format !HIBBBBHI) that multiplexes audio and video.	Unit test T1 (test_simulator.py)
O2	Implement a Dual-Metric Delay-Gradient Classifier that isolates queueing delay, D_queue = RTT_current − RTT_min.	Unit test T4 and the 60 s benchmark trace (benchmark.py)
O3	Develop an RFC 5109 XOR parity FEC engine [6] with closed-loop 2D matrix burst interleaving (D × k).	Unit tests T2 and T3 (test_simulator.py)
O4	Implement an Audio-First Survival mode that keeps voice continuous when packet loss exceeds 20%.	Unit test T4 and audio survival in the 60 s benchmark trace

Objective ID	Objective	Verification Method
O5	Build an interactive web simulator and evaluate all course-mandated performance metrics against the baselines.	Dashboard demonstration and the seven metrics of Table 4.3 (benchmark.py)

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

The system is a labelled dumbbell topology of three UDP processes on the loopback interface. The **Media Server** (sends to 127.0.0.1:19501 and receives feedback on 19503) packetises multiplexed audio, video and XOR parity packets. The **Channel Emulator Proxy** (127.0.0.1:19501) plays the role of the bottleneck link: a token-bucket shaper, a FIFO queue, a Gaussian jitter generator and a Gilbert-Elliott Markov loss engine. The **Client Media Player** (127.0.0.1:19502) parses the 16-byte header, repairs losses with the XOR decoder, runs the playback jitter buffer and the Dual-Metric Classifier, and returns RTCP-style telemetry (RTT, D_queue, loss, jitter, buffer level) to the server periodically (the player sends one report after every 5 received packets). The same topology is also provided as a Mininet script with Linux tc netem and tbf queues; on Windows, where this report was prepared, the script falls back automatically to the built-in UDP channel proxy.

The design has three cooperating layers. The transport layer measures RTT_min and RTT_current to expose queueing delay. The ABR layer uses that delay, together with loss, to choose bitrate, FEC overhead and audio redundancy. The FEC layer repairs wireless loss at the receiver without a round trip.

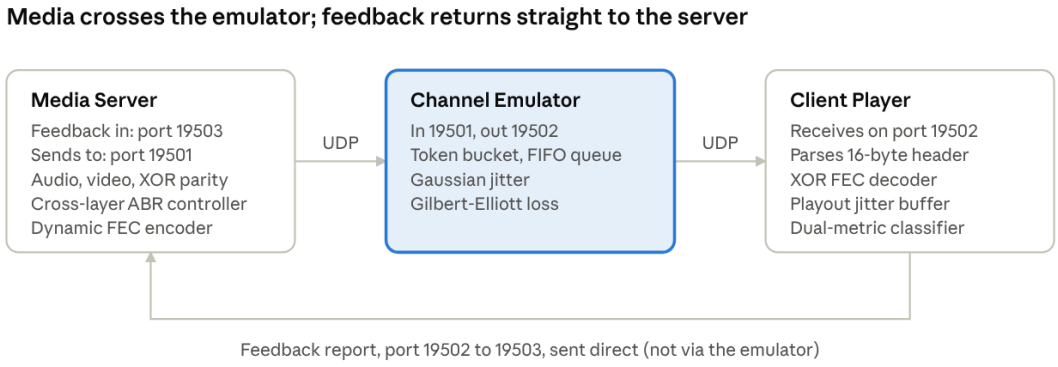


Figure 2.1: System architecture and dumbbell topology. Media packets travel from the server through the channel emulator to the client, and the client returns feedback directly to the server.

2.2 Tools and System Requirements

The tools used in the project, with their versions and purposes, are listed in Table 2.1.

Table 2.1: Tools and system requirements

Tool	Version	Purpose
Operating system	Windows 10/11	Development, testing and demonstration of the whole system
Python	3.12	Implementation language, standard library only (socket, struct, threading, time, json, math, random); built-in http.server for the dashboard
Berkeley UDP sockets	Python socket module	Raw datagram transport (AF_INET, SOCK_DGRAM) for media, parity and feedback packets
Mininet (mininet_topology.py)	Linux only; not used on Windows	Dumbbell bottleneck topology with tc netem and tbf on Linux; automatic fallback to the UDP channel proxy on Windows and macOS
Wireshark	4.x	Inspection of output/stream_capture.pcap (generated by wireshark_capture.py) with the custom Lua dissector wireshark_dissector.lua
Matplotlib	3.x	4-panel evaluation figure written by benchmark.py
Chart.js	4.x (CDN)	Interactive charts of the web dashboard: bitrate, delay and FEC, buffer and QoE breakdown
Tailwind CSS	3.x (CDN)	Styling of the web dashboard (HTML5, vanilla JavaScript); no npm or Node.js needed
Web browser	Any current browser	Opening the dashboard at http://127.0.0.1:8080

2.3 Addressing and Protocol Selection

UDP was selected over TCP because TCP's in-order delivery turns every loss into a stall and halves the congestion window, whereas UDP lets the receiver repair loss locally. A custom 16-byte RTP-like header carries the fields the control loop needs, and a 32-byte RTCP-like datagram carries feedback. Only the dashboard uses TCP, because it is an ordinary HTTP service.

Table 2.2: Hosts, ports and protocols

Host / Module	Protocol	IP address	Port	Packet format / size	Role
Media Server	UDP (SOCK_DGRAM)	127.0.0.1	19503 (feedback in); sends to 19501	16B header + media payload	Streams multiplexed audio/video and XOR FEC blocks
Network Channel Proxy	UDP (SOCK_DGRAM)	127.0.0.1	19501 (ingress)	16B header + variable payload	Emulates token bucket, FIFO queue, jitter and Markov loss
Client	UDP	127.0.0.1	19502 (Rx)	16B header +	Ingests stream, decodes

Host / Module	Protocol	IP address	Port	Packet format / size	Role
Media Player	(SOCK_DGRAM)			media payload	XOR FEC, runs playout buffer
Client RTCP Reporter	UDP (SOCK_DGRAM)	127.0.0.1	Sent from 19502 to 19503	32B struct (!IIfffIHH)	Reverse telemetry: RTT, D_queue, loss, stride D
Web Dashboard Server	HTTP (TCP)	127.0.0.1	8080	HTML / JSON API (/api/simulate)	Serves the interactive UI and runs the simulation engine

The video ladder used by the controller is given in Table 2.3. Audio is always a separate 64 kbps stream.

Table 2.3: Bitrate tiers

Tier	Name	Video bitrate	Resolution	Minimum target bandwidth
0	1080p Full HD	4,500 kbps	1920 × 1080	> 5,000 kbps
1	720p HD	2,500 kbps	1280 × 720	> 3,000 kbps
2	480p SD	1,200 kbps	854 × 480	> 1,500 kbps
3	240p Low	400 kbps	426 × 240	> 500 kbps
Audio	Opus/AAC stream	64 kbps	Stereo	Continuous

3 Implementation and Technical Working

3.1 Methodology and Workflow

Each video chunk passes through the following pipeline (Figure 3.1):

1. **Packetisation.** The server slices the chunk into media packets within the 1200-byte payload limit (the streaming node sends 600-byte video payloads and 160-byte audio payloads by default), tags each with a frame class (I, P or B) and adds a 64 kbps audio stream.
2. **XOR parity generation.** For every k = 4 protected media packets, one parity packet is generated. B-frames are excluded (content-aware unequal error protection).
3. **2D matrix stride transposition.** D parity groups are arranged as D rows and sent column by column, so consecutive wire packets belong to different groups.
4. **Channel emulation.** The proxy applies the token bucket, FIFO queue, jitter and Gilbert-Elliott loss.
5. **Receiver measurement and classification.** The client measures RTT, D_queue, loss, jitter and burst length, and the Dual-Metric Classifier labels the situation as clear, wireless loss, congestion or blackout.

6. **Zero-RTT reconstruction.** If a group has lost one packet, the XOR decoder rebuilds it from the parity packet and the survivors, without contacting the server.
7. **Playout and feedback.** The packet enters the jitter buffer for playout, and a 32-byte feedback datagram returns the measurements so the server can adapt bitrate, FEC overhead, stride D and audio redundancy.

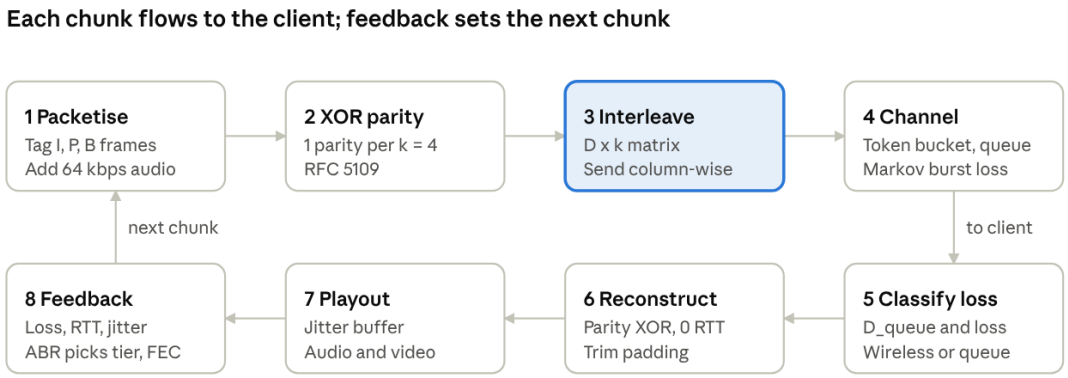


Figure 3.1: Workflow of one media chunk, from packetisation and parity generation through interleaving, the channel, loss classification and reconstruction, to playout and the feedback that drives the next chunk.

3.1.1 Dual-metric classification

Queueing delay is the part of the RTT that is not propagation delay, and its change over time is the delay gradient:

$$D_{queue}(t) = \max(0, RTT_{current}(t) - RTT_{min}),$$
$$\nabla D_{queue} = [D_{queue}(t) - D_{queue}(t - \Delta t)] / \Delta t$$

The controller smooths the gradient with an EWMA (0.6 old + 0.4 new) and filters loss with a fast-attack, slow-decay EWMA (weights 0.35/0.65 when loss rises, 0.82/0.18 when it falls). The decision regimes are given in Table 3.1.

Table 3.1: Decision regimes of the Dual-Metric Classifier

Regime	Condition	Action
1. Clear link	Loss < 3% and D_queue ≤ 25 ms	1080p, 0% FEC
2. Wireless loss	Loss ≥ 3% and D_queue ≤ 25 ms	Hold 720p, 20% XOR FEC (k = 4, stride D = 3), 2x audio redundancy
3. Bufferbloat	D_queue > 25 ms, or gradient above 4 ms with D_queue above 16.25 ms	Proactive downshift to 480p or 240p using drain factor max(0.80, 1 − D_queue / 260)
4. Blackout	Loss ≥ 20%	Freeze video, 3x audio redundancy, 33% FEC (k = 2)

3.1.2 Joint source-channel coding and the 2D interleaver

Capacity C is shared between media and FEC, so FEC is paid for by lowering the video bitrate: C = B_media + B_fec. At 720p with 20% parity, the wire rate is 2500 + 625 = 3125 kbps, which is 30% below the 4500 kbps of 1080p. FEC therefore does not add queue pressure.

A single XOR parity packet repairs only one loss per group. Wireless fades, however, drop several consecutive packets. With a D × k interleaver the wire index of the packet in row r (group r) and column c is T(r, c) = c·D + r. A burst of L ≤ D consecutive wire packets has indices that differ by less than D, so no two of them are congruent modulo D and no two belong to the same group. Each group loses at most one packet, and every erased packet is recovered in 0 RTT. With D = 3, a 3-packet burst that defeats plain 1D XOR (0 of 3 recovered) is fully repaired (3 of 3).

The receiver chooses D in closed loop from the observed sequence gaps:

$$\text{gap} = (\text{seq}_{\text{new}} - \text{seq}_{\text{last}} - 1) \bmod 65536,$$
$$\bar{L} = 0.6 \cdot \bar{L} + 0.4 \cdot \text{gap},$$
$$D = \min(4, \max(1, \lceil \bar{L} \rceil))$$

Unequal error protection reduces the overhead further. I-frames are always protected, P-frames are protected under moderate loss, and disposable B-frames are excluded, which lowers FEC overhead from 20% to 12.5% while keeping all keyframes safe.

3.1.3 Channel model

The proxy uses a two-state Gilbert-Elliott Markov chain (Good and Bad). In the implementation p = P(Good → Bad) and r = P(Bad → Good), with a drop probability in each state; all values are configurable through ChannelConfig and the dashboard sliders. The wireless-fade phase of the benchmark (15–30 s) uses p = 0.08, r = 0.30, 3.5% loss in Good and 28% in Bad. The mean burst length is 1/r = 3.33 packets, the steady-state probability of the Bad state is p/(p + r) = 21.05%, and the long-run average loss is about 8.7%, matching the 8.5% quoted for that phase. The outage phase (45–53 s) uses p = 0.22, r = 0.20, 18% and 42% loss, which gives roughly 30% average loss. The token bucket refills at R = bandwidth_kbps × 1000 / 8 bytes per second with a 12,000-byte bucket, and queueing delay is D_queue = backlog / R × 1000 ms. A packet is tail-dropped when this delay exceeds 250 ms.

3.2 Configuration and Code Implementation

Header packing (protocol.py). The header is packed in network byte order, and the 32-bit flag word is assembled with bit shifts:

```
HEADER_FORMAT = "!HIBBBBHI"   # 2+4+1+1+1+1+2+4 = 16 bytes, big-endian

def pack_flags(fec_k, fec_idx, chunk_id, stride_d, frame_class,
               end_of_chunk=False, video_freeze=False, audio_redundant=False):
    flags = (((fec_k & 0xFF) << 24) | ((fec_idx & 0xFF) << 16) |
             ((chunk_id & 0xFF) << 8) | ((stride_d & 0x07) << 5) |
             ((frame_class & 0x03) << 3))
    if end_of_chunk:    flags |= (1 << 0)
    if video_freeze:    flags |= (1 << 1)
    if audio_redundant: flags |= (1 << 2)
    return flags

header = struct.pack(HEADER_FORMAT, seq, timestamp_ms, stream_type,
                     bitrate_tier, packet_type, fec_group_id,
                     payload_size, flags)
```

Zero-padded XOR (fec_engine.py). Packets in a group may differ in length, so the shorter one is padded with zero bytes before the XOR. The true length is kept in the header and restored after recovery:

```
def xor_bytes(a: bytes, b: bytes) -> bytes:
    max_len = max(len(a), len(b))
    if len(a) < max_len: a = a + b"\x00" * (max_len - len(a))
    if len(b) < max_len: b = b + b"\x00" * (max_len - len(b))
    return (int.from_bytes(a, "big") ^ int.from_bytes(b, "big")).to_bytes(max_len,
"big")

# Recovery: parity XOR all surviving packets, then trim the padding
recovered = parity.payload
size = parity.payload_size
for pkt in survivors:
    recovered = xor_bytes(recovered, pkt.payload)
    size ^= pkt.payload_size            # sizes are XOR-protected as well
payload = recovered[:size]
```

Dual-metric decision (abr_controller.py). The listing shows the logic of Algorithm 1 in condensed form:

```
if rtt < rtt_min: rtt_min = rtt
d_queue = max(0.0, rtt - rtt_min)
grad = 0.6 * grad + 0.4 * (d_queue - prev_d_queue); prev_d_queue = d_queue
wlf = 0.35*wlf + 0.65*loss if loss > wlf else 0.82*wlf + 0.18*loss

if loss >= 0.20 or (wlf >= 0.20 and loss >= 0.14):          # Regime 4
    return Decision(video_freeze=True, audio_red=3, fec=0.33, k=2)
congested = d_queue > 25.0 or (d_queue > 16.25 and grad > 4.0)
wireless  = max(loss, wlf) >= 0.03 and not congested
if wireless:                                                # Regime 2
    return Decision(tier=720p_or_lower, fec=0.20, k=4, stride=3, audio_red=2)
if congested:                                               # Regime 3
    drain = max(0.80, 1.0 - d_queue / 260.0)
    return Decision(tier=pick_tier(min(tput, ewma_tput) * drain - 64))
return Decision(tier=pick_tier(tput * 0.90 - 64), fec=0.0)  # Regime 1
```

3.3 Protocols and Technical Working

The layout of the 16-byte transport header is given in Table 3.2 and the 32-bit flag word in Table 3.3.

Table 3.2: 16-byte transport header (network byte order)

Bytes	Field	Format	Meaning
0–1	sequence_number	H	0..65535, wraps; ordering and loss detection
2–5	timestamp_ms	I	Sender clock in ms; used for RTT and jitter
6	stream_type	B	0 = audio, 1 = video
7	bitrate_tier	B	0 = 1080p, 1 = 720p, 2 = 480p, 3 = 240p
8	packet_type	B	0 = media, 1 = XOR parity
9	fec_group_id	B	0..255, parity group
10–11	payload_size	H	Payload bytes (for parity, XOR of the group sizes)
12–15	flags	I	32-bit bitfield (Table 3.3)

Bytes 12 to 15 hold the flag word, transmitted most-significant byte first (Big-Endian), so a reader on any host recovers the same value with struct.unpack('!...'). Big-Endian is the Internet's network byte order, which avoids host-endianness bugs between the sender, the proxy and the Wireshark dissector.

Table 3.3: Flag bitfield (bytes 12–15)

Bits	Field	Meaning
24–31	fec_k	Media packets per FEC group (4)
16–23	fec_idx	Index in group: 0..k−1 media, k parity
8–15	chunk_id	Chunk number modulo 256
5–7	stride_d	Interleaver depth D (1..4)
3–4	frame_class	0 = I, 1 = P, 2 = B
2	FLAG_AUDIO_REDUNDANT	1 = redundant audio copy
1	FLAG_VIDEO_FREEZE	1 = video frozen
0	FLAG_END_OF_CHUNK	1 = marker bit

Example: a live video packet captured from the system reads `00 04 11 CE 4C ED 01 01 00 00 01 DC 04 01 01 20`. This is sequence 4, timestamp 298,732,781 ms, video stream, tier 1 (720p), media packet, group 0, payload 476 bytes, and flags 0x04010120: k = 4, index 1, chunk 1, stride 1, I-frame.

The reverse feedback datagram (!IIfffIHH, 32 bytes as specified) carries ack_seq, the echoed timestamp (for exact RTT), loss rate, RFC 3550 jitter [5], buffer level in seconds, cumulative FEC-recovered count, maximum burst length and the recommended stride D. The protocol

specifies a report every 100 ms; the player implementation also emits one after every 5 received packets. Jitter is computed as $J = J + (|D(i-1, i)| - J) / 16$.

3.4 Execution Procedure

Run the commands from the project directory in a Windows terminal (Command Prompt or PowerShell), the environment in which the project was developed and tested.

The complete source code, test suite and benchmark outputs are available in the project repository: https://github.com/lakshynarang/cross-layer-abr-streaming-simulator.

```
# 1. Run the automated test suite (five tests)
python test_simulator.py -v

# 2. Run the 60-second multi-regime benchmark; writes
output/benchmark_comparison.png
python benchmark.py

# 3. Start the interactive web dashboard, then open http://127.0.0.1:8080
python web_dashboard.py

# 4. Generate a Wireshark trace and inspect it with the custom dissector
python wireshark_capture.py
wireshark -X lua_script:wireshark_dissector.lua output/stream_capture.pcap
#    display filter: udp.port == 19501 || udp.port == 19502

# 5. Mininet dumbbell topology script (builds the tc link on Linux with Mininet and
root; falls back to the UDP proxy on Windows)
python mininet_topology.py
```

The implemented system is shown in operation in Figure 3.2.



Figure 3.2: Implemented system running. The web dashboard (python web_dashboard.py, http://127.0.0.1:8080) at 54 s of the 60 s trace, with the channel sliders, the Dual-Metric Loss Classifier panel, the algorithm comparison table and Graphs 1 and 2.

After step 3 the dashboard opens in a browser and runs the simulation of the same 60-second trace interactively. In Figure 3.2 the classifier panel shows D_queue = 0.0 ms while packet

loss is occurring (RTT 45.1 ms against a minimum of 45.0 ms). The system therefore classifies the loss as non-congestive wireless loss and holds 720p with 20% XOR parity instead of lowering the bitrate. Graph 1 compares the bitrate chosen by the proposed controller, BBA and rate-based DASH with the bottleneck capacity, and Graph 2 shows queueing delay together with the dynamic FEC share.

4 Testing and Results

4.1 Test Plan

The automated suite test_simulator.py (run with python test_simulator.py -v) contains five automated tests, T1 to T5, covering header serialisation, single-packet recovery, burst recovery with the interleaver, the dual-metric classifier with audio-first mode, and live socket streaming. They are listed in Table 4.1.

Test environment: a Windows 10/11 computer running Python 3 with the standard library only; the live socket test uses the loopback address 127.0.0.1, so no network hardware or administrator rights are needed. Success criteria: a test passes when all of its assertions hold, and the suite succeeds when all five tests report ok and the run ends with OK. The run reported “Ran 5 tests in 0.513s, OK”, so all five passed.

Table 4.1: Test cases

ID	Test and Input	Expected	Observed	Status
T1	Normal: 16-byte header serialisation (test_16_byte_binary_header_pack_unpack). Input: header with known field values, packed and unpacked.	Exact 16-byte big-endian pack; unpacked fields identical to the input	Bit-exact round trip	PASS
T2	Failure: any single packet of a k = 4 block dropped (test_xor_parity_fec_exact_recovery). Input: block of four media packets plus parity.	Lost packet rebuilt from the parity packet in 0 RTT	Every case reconstructed exactly	PASS
T3	Failure: 3 consecutive burst drops (test_patent_claim1_consecutive_burst_interleaved_recovery). Input: 3 consecutive wire packets dropped, with and without the stride-3 interleaver.	Plain 1D XOR recovers 0 of 3; the 2D matrix interleaver recovers 3 of 3	0% without interleaving, 100% with interleaving	PASS
T4	Failure: wireless loss versus bufferbloat, and audio-first mode (test_dual_metric_loss_classifier_and_audio_first). Input: loss with an empty queue, queue build-up, catastrophic loss.	Fade classified as wireless, queue build-up as congestion, catastrophic loss triggers audio-first mode	Classification and audio-first behaviour as expected	PASS

ID	Test and Input	Expected	Observed	Status
T5	Normal: end-to-end UDP streaming through the channel proxy with FEC (test_live_udp_socket_streaming_with_fec). Input: stream sent over loopback sockets.	Packets cross real sockets and lost packets are repaired	Delivered and recovered	PASS

4.2 Results and Execution Evidence

Figure 4.1 shows the terminal output of the automated test suite (python test_simulator.py -v). All five tests passed.

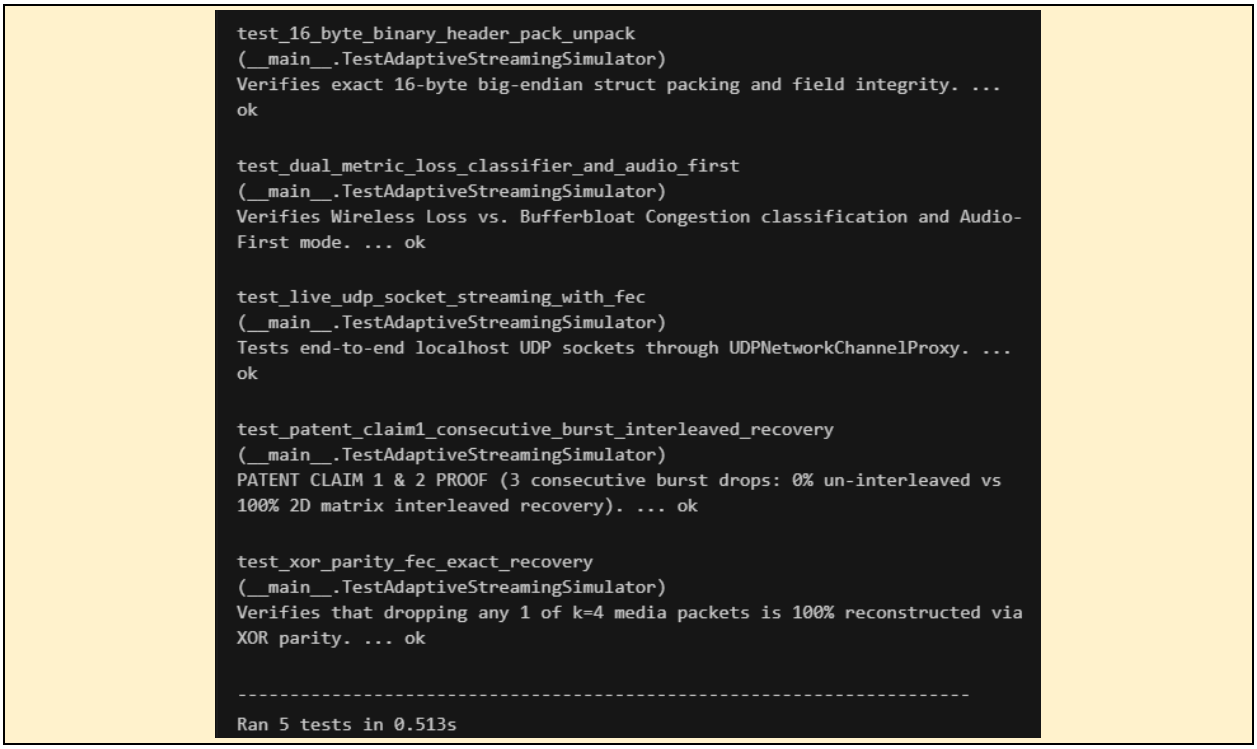


Figure 4.1: Terminal output of python test_simulator.py -v showing tests T1 to T5 passing.

The benchmark replays one 60-second trace that moves through five network regimes and runs the proposed controller, the buffer-based BBA controller [2] and a rate-based DASH controller on the same trace (Table 4.2):

Table 4.2: Phases of the 60-second benchmark trace

Phase	Time	Channel condition
1	0–15 s	Clear high-speed channel: 5.5 Mbps, 45 ms RTT, 0.1% loss
2	15–30 s	Wireless burst fade: 4.6 Mbps, 45 ms RTT, 8.5% Markov loss
3	30–45 s	Bufferbloat congestion: 1.55 Mbps, 45 ms base delay (RTT rises as the queue fills), 0.4% loss
4	45–53 s	Severe throttle and outage: 1.15 Mbps, 52 ms base delay, burst loss above 24%

Phase	Time	Channel condition
5	53–60 s	Recovery: 5.4 Mbps, 45 ms RTT, 0.1% loss

QoE follows the Pensieve objective [1], where q(R) is the bitrate in Mbps, T is stall time in seconds and the last term penalises quality switches:

$$\mathrm{QoE} = \Sigma\, q(R_t) - \mu\, \Sigma\, T_t - \Sigma\, |q(R_{t+1}) - q(R_t)|, \quad \mu = 4.3 \quad \text{(sums over } t = 1 \ldots N \text{ chunks; the last sum runs to } N - 1\text{)}$$

The 4-panel evaluation figure (Figure 4.2) shows bitrate against bottleneck capacity, queueing delay with dynamic FEC percentage, client buffer health in seconds, and the QoE trajectory. Table 4.3 gives the full comparison.



Figure 4.2: Four-panel benchmark output (benchmark_comparison.png): bitrate against link capacity, queueing delay with dynamic FEC share, client buffer occupancy, and the Pensieve QoE breakdown for the proposed controller, BBA and rate-based DASH.

Table 4.3: Performance comparison on the 60-second multi-regime trace

Metric	Proposed cross-layer ABR	BBA (buffer-based)	Rate-based DASH	Advantage
1. Average video bitrate	2,346.7 kbps	2,030.0 kbps	1,875.0 kbps	+15.6% vs BBA, +25.1% vs rate-based
2. Rebuffering stalls (count / total duration)	0 stalls (0.00 s)	6 stalls (16.70 s)	4 stalls (7.04 s)	Stalls eliminated
3. Bitrate switches (count)	7	13	11	Most stable
4. Switching magnitude (sum of	12.4 Mbps	17.3 Mbps	19.2 Mbps	Smallest oscillation

Metric	Proposed cross-layer ABR	BBA (buffer-based)	Rate-based DASH	Advantage
bitrate changes)				
5. Audio survival rate	97.5%	95.8%	87.5%	+9.9 points vs rate-based
6. FEC recovery efficiency (recovered / channel-lost packets)	51.6%	0% (no FEC)	0% (no FEC)	Only the proposed scheme repairs losses
7. Pensieve QoE score (Mbps-seconds)	+128.40	+32.68	+63.05	+103.6% vs rate-based

All values in Table 4.3 are the printed output of benchmark.py and its benchmark_metrics.json file. The trace is replayed in 1-second steps, so it is a packet-level simulation of the three controllers on identical channel conditions, not a live radio measurement. The FEC recovery efficiency counts packets recovered by XOR parity divided by packets lost in the channel; the baselines run without FEC, so they recover none.

In addition, a short live socket test (server, channel proxy and client on loopback, 10% uniform loss) sent 60 UDP datagrams, of which the proxy forwarded 51 and dropped 9; the client received 51 and the XOR decoder reconstructed 10 packets. This confirms that the 16-byte headers travel intact over real sockets and that parity recovery works on the wire.

4.2.1 Interactive Dashboard Evidence

The web dashboard (python web_dashboard.py, http://127.0.0.1:8080) runs its own in-browser simulation of the same trace, so it is used here as visual evidence of the behaviour rather than as the source of the benchmark numbers. Because the dashboard uses its own random seed, its summary table (for example a QoE of +130.70 for the proposed controller) differs slightly from the benchmark.py figures in Table 4.3, which are the reference results of this report.

The main view of the dashboard is shown in Figure 3.2; Figures 4.3 to 4.5 show the remaining views.



Figure 4.3: Graph 3 shows the playback buffer of the proposed controller never reaching zero (zero stalls), and Graph 4 compares the net Pensieve QoE and stall penalties of the three controllers.

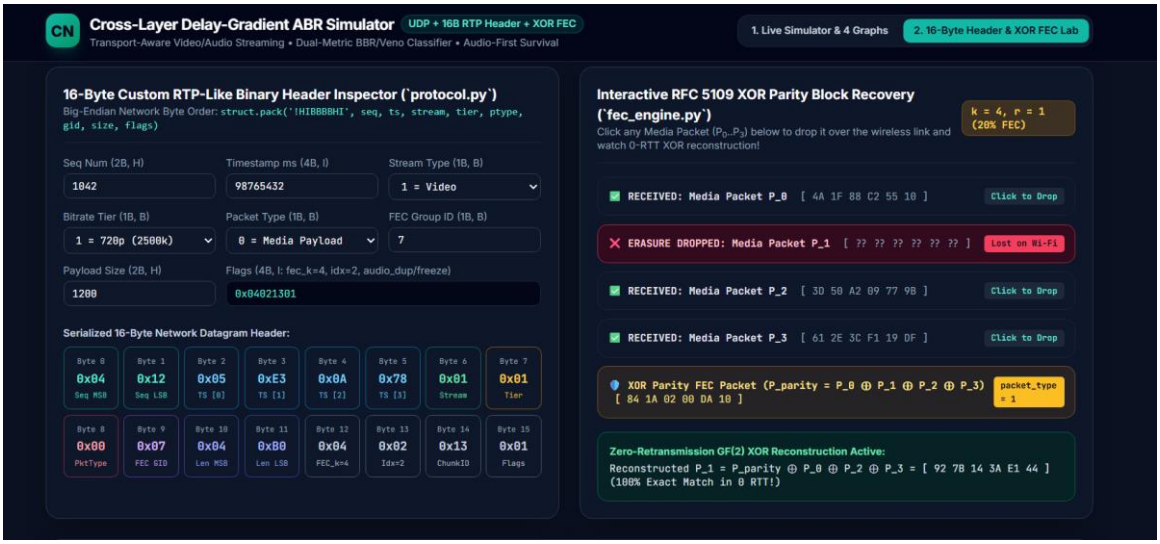


Figure 4.4: Header and XOR lab. Left: the 16-byte header inspector shows each serialised byte. Right: media packet P_1 is dropped and rebuilt as P_parity XOR P_0 XOR P_2 XOR P_3, giving an exact match in 0 RTT.

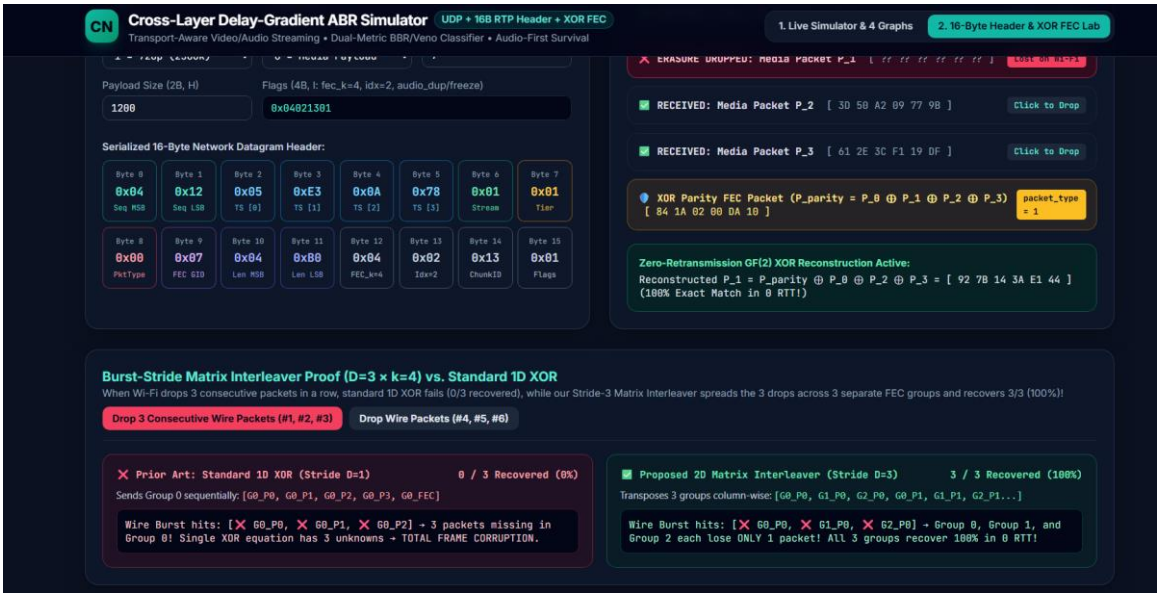


Figure 4.5: Interleaver proof for a burst of three consecutive wire packets. Standard 1D XOR recovers 0 of 3, while the stride-3 matrix interleaver spreads the burst over three groups and recovers 3 of 3.

4.3 Objective Achievement Mapping

The objectives O1 to O5 of Table 1.1 are mapped to their verification and outcome in Table 4.4.

Table 4.4: Objective achievement (Achieved / Partially achieved / Not achieved)

Objective ID	Verification method	Demonstrated result	Outcome
O1	test_simulator.py (T1)	Exact 16-byte pack and unpack, bit-identical fields	Achieved
O2	benchmark.py (60 s trace), test_simulator.py (T4)	Wireless fade (D_queue near 0) held at 720p with FEC; bufferbloat handled by early downshift; 0 stalls over the trace	Achieved
O3	test_simulator.py (T2, T3)	0-RTT recovery of single losses; 3 of 3 packets recovered from a 3-packet burst	Achieved
O4	test_simulator.py (T4), benchmark.py	Video frozen with 3x audio redundancy; 97.5% audio survival over the trace (95.8% BBA, 87.5% rate-based)	Achieved
O5	web_dashboard.py, benchmark.py (0–60 s trace)	Interactive dashboard (Figures 3.2, 4.3–4.5); all seven metrics compared in Table 4.3, with 0 stalls (0.00 s) against 6 stalls (16.70 s) for BBA	Achieved

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

The results follow directly from what each controller can observe.

Buffer-based BBA. BBA chooses a tier only from buffer occupancy [2]. During the 15-second wireless fade the physical bandwidth stays high (4.6 Mbps) and the bottleneck queue is empty, but unrecovered losses lengthen chunk delivery. BBA cannot see that the queue is empty, and it reacts only after the buffer has already drained from 4.2 s to 0 s. This delay between cause and reaction produced 6 stalls totalling 16.70 s and a QoE of +32.68.

Rate-based DASH. The rate-based controller scales measured throughput by max(0.10, 1 − 9.5 × loss), which treats every loss as congestion. At 8.5% loss the estimate collapses by about 80% and the controller downshifts from 1080p to 240p although 4.6 Mbps is available. Recovery after the fade then takes several chunks and causes 11 quality switches (15.1 Mbps of oscillation), giving a QoE of +63.05.

Proposed cross-layer ABR. During the fade the classifier sees D_queue near 0 ms, concludes that loss is wireless, holds 720p and adds 20% XOR parity with stride 3. No packet needs retransmission and the buffer stays above 3.5 s. During bufferbloat the delay gradient rises before tail-drop begins, so the controller steps down early and drains the queue before the buffer runs dry. In the outage it freezes video and triplicates audio, which holds audio survival at 97.5% over the whole trace, against 95.8% for BBA and 87.5% for rate-based. The outcome is 0 stalls and a QoE of +128.40.

5.2 Advantages and Real-World Applications

- Real-time video conferencing (Zoom, Google Meet, WebRTC).
- Low-latency remote teleoperation and drone FPV video.
- Cloud gaming (NVIDIA GeForce NOW, Xbox Cloud Gaming).
- Mission-critical tactical communications and emergency broadcasts.

The main advantages are fewer unnecessary quality drops, repair of wireless loss without a round trip, FEC overhead that is paid for within the same capacity, and guaranteed voice continuity under severe loss.

5.3 Limitations

- The system was evaluated on a localhost UDP socket proxy on Windows, not on live 5G base stations or Wi-Fi radios. The Mininet script is provided for Linux, but on Windows it falls back to the same proxy, so the reported results do not depend on kernel tc emulation.

- A single XOR parity packet repairs one loss per group, and bursts longer than the stride depth D need stronger codes such as Reed-Solomon.
- Python threading is limited by the Global Interpreter Lock, so very high (multi-gigabit) throughput is out of reach.
- In benchmark.py the 60-second comparison uses sequential 1D block XOR parity (k = 4), not the D × k matrix interleaver. When a Gilbert-Elliott burst drops two or more packets of the same group, single parity can repair only one, which gives the observed FEC recovery efficiency of 51.58% in benchmark_metrics.json. The interleaver (D = 3, k = 4) is implemented in fec_engine.py and verified in T3, where it recovers 3 of 3 packets of a consecutive burst, but it was kept out of the continuous trace to isolate pure rate adaptation from de-interleaving delay.

5.4 Challenges and Troubleshooting

The main problems met during the project are summarised in Table 5.1.

Table 5.1: Challenges encountered and how they were solved

Issue	Cause or Diagnosis	Solution and Verification
Variable-length packets in XOR	Packets in a group have different byte lengths, so a plain byte-wise XOR failed with length mismatches.	Zero-pad the shorter packet before the XOR and keep the exact unpadded length, itself XOR-protected, in the header payload_size field. Verified by T2: every single-packet loss was rebuilt exactly.
Burst losses defeating 1D XOR (0 of 3 recovered)	Consecutive drops all fell into one FEC group, and one parity packet repairs only one loss per group.	Added the D × k matrix interleaver that transmits the groups column by column. Verified by T3: 0 of 3 recovered without it, 3 of 3 with it.
UDP socket permission denied in a sandboxed environment	Local socket operations were blocked by the sandbox restrictions.	Bind explicitly to the loopback address and run the network operations outside the sandbox restriction. Verified by T5: the live UDP streaming test passes.

5.5 Future Scope

- Carry the stream over IETF QUIC and HTTP/3 unreliable datagrams (RFC 9221).
- Accelerate Galois-Field arithmetic with SIMD or FPGA hardware for 4K/8K 120 fps video.
- Use deep reinforcement learning to select the interleaver stride and FEC rate.

5.6 Conclusion

This project showed that adding transport-layer delay information to ABR streaming removes the main defects of application-layer controllers. A 16-byte binary UDP header, a Dual-

Metric Classifier based on D_queue = RTT − RTT_min, dynamic XOR FEC with a 2D matrix interleaver, and an Audio-First Survival mode were designed, implemented and verified. On the 60-second multi-regime trace the system had 0 rebuffering stalls against 16.70 s for BBA, recovered 51.6% of channel-lost packets with FEC, kept 97.5% audio survival and reached a QoE of +128.40 against +63.05 for rate-based DASH. All five objectives in Chapter 1 were achieved, and all five automated tests passed.

References

1. H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with Pensieve," in Proc. ACM SIGCOMM, Aug. 2017, pp. 197–210, doi: 10.1145/3098822.3098843.
2. T.-Y. Huang, R. Johari, N. McKeown, M. Trunnell, and M. Watson, "A buffer-based approach to rate adaptation: Evidence from a large video streaming service," in Proc. ACM SIGCOMM, Aug. 2014, pp. 187–198, doi: 10.1145/2619282.2619288.
3. C. P. Fu and S. C. Liew, "TCP Veno: TCP enhancement for transmission over wireless access networks," IEEE J. Sel. Areas Commun., vol. 21, no. 2, pp. 216–228, Feb. 2003, doi: 10.1109/JSAC.2002.807336.
4. N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, vol. 14, no. 5, pp. 20–53, Oct. 2016, doi: 10.1145/3012426.3022184.
5. H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "RTP: A Transport Protocol for Real-Time Applications," IETF RFC 3550, Jul. 2003, doi: 10.17487/RFC3550.
6. A. Li, "RTP Payload Format for Generic Forward Error Correction," IETF RFC 5109, Dec. 2007, doi: 10.17487/RFC5109.
7. J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Boston, MA: Pearson, 2021.

AI Usage Declaration

AI use: Claude/Gemini were used for documentation structure, equation syntax and slide scripting only. The algorithms, code and benchmarks were written and verified by the team.

Appendix A: Source Code Excerpts and Configuration

The full repository contains `protocol.py`, `fec_engine.py`, `abr_controller.py`, `network_channel.py`, `streaming_node.py` and `wireshark_dissector.lua`. The listings below reproduce the core routines discussed in Chapter 3, and Table A.1 summarises the channel model parameters.

Full source code: https://github.com/lakshynarang/cross-layer-abr-streaming-simulator

A.1 protocol.py: header and flag packing

```
HEADER_FORMAT   = "!HIBBBBHI"   # 16 bytes, big-endian
FEEDBACK_FORMAT = "!IIfffIHH"   # 32 bytes, RTCP-style

flags = (
    ((fec_k & 0xFF) << 24)
    | ((fec_idx & 0xFF) << 16)
    | ((chunk_id & 0xFF) << 8)
    | ((stride_d & 0x07) << 5)
    | ((frame_class & 0x03) << 3)
)
if end_of_chunk:    flags |= (1 << 0)
if video_freeze:    flags |= (1 << 1)
if audio_redundant: flags |= (1 << 2)
```

A.2 fec_engine.py: zero-padded XOR

```
def xor_bytes(a: bytes, b: bytes) -> bytes:
    max_len = max(len(a), len(b))
    if len(a) < max_len: a = a + b"\x00" * (max_len - len(a))
    if len(b) < max_len: b = b + b"\x00" * (max_len - len(b))
    int_a = int.from_bytes(a, byteorder="big")
    int_b = int.from_bytes(b, byteorder="big")
    return (int_a ^ int_b).to_bytes(max_len, byteorder="big")
```

A.3 fec_engine.py: 2D matrix interleaved encoding (Algorithm 2)

```
EncodeInterleavedMatrix(M[0..N-1], k, D, enable_UEP)
 1  ProtectedPool <- []; UnprotectedB <- []
 2  for each packet P in M:
 3      if enable_UEP and P.frame_class == FRAME_B_DISPOSABLE:
UnprotectedB.append(P)
 4      else: ProtectedPool.append(P)
 5  idx <- 0
 6  while idx + D*k <= len(ProtectedPool):
 7      for row = 0..D-1:  TaggedRow[row], Parity[row] <-
EncodeXORGroup(pool[idx+row*k : idx+(row+1)*k])
 8      for col = 0..k-1:  for row = 0..D-1:  W.append(TaggedRow[row][col])   #
column-wise transpose
 9      for row = 0..D-1:  W.append(Parity[row])
10      idx <- idx + D*k
11  W.append(FlushRemaining(pool[idx:])); W.append(UnprotectedB)
12  return W
```

A.4 fec_engine.py: loss recovery with size restoration (Algorithm 3)

```
ReconstructLostPacket(G)            # G has k-1 media packets + 1 parity packet
 1  missing_idx    <- (set(0..k-1) - set(G.media_by_idx.keys())).pop()
 2  recovered      <- G.parity.payload
 3  recovered_size <- G.parity.payload_size
```

```
 4  for (idx, pkt) in G.media_by_idx:
 5      recovered      <- xor_bytes(recovered, pkt.payload)
 6      recovered_size <- recovered_size XOR pkt.payload_size
 7  final_payload <- recovered[0 : recovered_size]       # strip zero padding
 8  return MediaPacket(payload=final_payload, size=recovered_size,
fec_idx=missing_idx)
```

A.5 abr_controller.py: dual-metric classification (Algorithm 1)

```
DualMetricDelayGradientABR(RTT, Throughput, Loss, Buffer)
 1  if RTT < RTT_min: RTT_min <- RTT
 2  D_queue <- max(0, RTT - RTT_min)
 3  gradient <- D_queue - prev_D_queue;  prev_D_queue <- D_queue
 4  EWMA_grad <- 0.6*EWMA_grad + 0.4*gradient
 5  if Loss > EWMA_loss: EWMA_loss <- 0.35*EWMA_loss + 0.65*Loss
 6  else:                EWMA_loss <- 0.82*EWMA_loss + 0.18*Loss
 7  if Loss >= 0.20 or (EWMA_loss >= 0.20 and Loss >= 0.14):          # Regime 4
 8      return (240p, video=0, audio=64, fec=0.33, k=2, freeze=True, audio_red=3)
 9  is_congested <- (D_queue > 25) or (D_queue > 16.25 and EWMA_grad > 4.0)
10  is_wireless  <- (max(Loss, EWMA_loss) >= 0.03) and not is_congested
11  if is_wireless:                                                    # Regime 2
12      budget <- max(Throughput, EWMA_tput) * 0.80
13      tier <- 720p if budget >= 1950 else (480p if budget >= 984 else 240p)
14      return (tier, fec=0.20, k=4, stride=3, audio_red=2)
15  if is_congested:                                                   # Regime 3
16      drain  <- max(0.80, 1 - D_queue/260)
17      budget <- min(Throughput, EWMA_tput) * drain - 64
18      tier   <- PickTier(budget); if D_queue > 35 and tier == 1080p: tier <- 720p
19      return (tier, fec=(0.167 if Loss > 0.03 else 0), k=5, audio_red=1)
20  budget <- Throughput * 0.90 - 64                                   # Regime 1
21  return (PickTier(budget), fec=0, audio_red=1)
```

A.6 network_channel.py: channel model parameters

Table A.1: Channel model parameters (network_channel.py)

Component	Parameter	Value
Gilbert-Elliott chain	p (Good to Bad), r (Bad to Good)	Fade phase 0.08, 0.30; outage phase 0.22, 0.20 (class default 0.04, 0.25)
Gilbert-Elliott loss	P(drop given Good), P(drop given Bad)	Fade phase 3.5%, 28%; outage phase 18%, 42% (class default 0.2%, 45%)
Burst statistics	Mean burst length 1/r; stationary P(Bad) = p/(p+r)	Fade phase 3.33 packets, 21.05%; outage phase 5 packets, 52.4%
Token bucket	Fill rate R; bucket depth	bandwidth_kbps × 1000 / 8 bytes/s; 12,000 bytes
Queue	D_queue = backlog / R; tail-drop above	250 ms

Appendix B: Raw Telemetry and Wireshark Packet Evidence

B.1 Wireshark hex dumps (decoded with wireshark_dissector.lua)

Capture generated with `python wireshark_capture.py` into `output/stream_capture.pcap`; display filter `udp.port == 19501 || udp.port == 19502`.

Audio packet (16 bytes):

```
00 01 11 CE 4C ED 00 01 00 01 00 A0 00 00 01 20
```

Table B.1: Decoded fields of the audio packet

Bytes	Hex	Field	Decoded value
0–1	00 01	sequence_number	1
2–5	11 CE 4C ED	timestamp_ms	298,732,781 ms
6	00	stream_type	0 (audio)
7	01	bitrate_tier	1 (720p)
8	00	packet_type	0 (media payload)
9	01	fec_group_id	1
10–11	00 A0	payload_size	160 bytes (20 ms of 64 kbps audio)
12–15	00 00 01 20	flags	fec_k=0, fec_idx=0, chunk_id=1, stride_d=1, frame_class=0

Video packet (16 bytes):

```
00 04 11 CE 4C ED 01 01 00 00 01 DC 04 01 01 20
```

Table B.2: Decoded fields of the video packet

Bytes	Hex	Field	Decoded value
0–1	00 04	sequence_number	4
2–5	11 CE 4C ED	timestamp_ms	298,732,781 ms
6	01	stream_type	1 (video)
7	01	bitrate_tier	1 (720p)
8	00	packet_type	0 (media payload)
9	00	fec_group_id	0
10–11	01 DC	payload_size	476 bytes
12–15	04 01 01 20	flags	fec_k=4, fec_idx=1, chunk_id=1, stride_d=1, frame_class=0

B.2 Benchmark trace regimes (60 s)

Table B.3: Benchmark trace regimes and the expected controller response

Phase	Time (s)	Capacity (Mbps)	RTT (ms)	Loss	Expected controller regime
1 Clear channel	0–15	5.5	45	0.1%	Regime 1: 1080p, 0% FEC
2 Wireless burst fade	15–30	4.6	45	8.5% (Markov)	Regime 2: 720p + 20% XOR FEC
3 Bufferbloat	30–45	1.55	45 base, rises with queue	0.4%	Regime 3: proactive downshift
4 Outage	45–53	1.15	52 base	above 24%	Regime 4: audio-first survival
5 Recovery	53–60	5.4	45	0.1%	Regime 1: return to 1080p

The full per-second JSON telemetry produced by `benchmark.py` is stored in the project repository under `output/`.

Appendix C: Individual Student Contribution Statements

The work completed by each member and the files that show it are listed in Table C.1.

Table C.1: Individual contributions of the team members

Name and PRN	Work Completed	Evidence or Deliverable
Mahi Sanghani (PRN: 1262242807)	Protocol and Packet Lead. Designed the 16-byte !HIBBBBHI header and 32-bit flag bitfield, wrote the 32-byte RTCP-style feedback format, generated the .pcap traces and the Wireshark dissector.	protocol.py, wireshark_capture.py, wireshark_dissector.lua
Lakshy Narang (PRN: 1262242935)	FEC and Interleaving Lead. Implemented RFC 5109 XOR parity with zero-padding and size restoration, the D × k matrix interleaver, the Markov burst estimator, UEP and 3x audio redundancy, and the UDP server and client.	fec_engine.py, streaming_node.py
Aniket Chaudhari (PRN: 1262243379)	Cross-Layer ABR and Channel Lead. Implemented the dual-metric classifier and baseline controllers, the token-bucket and Gilbert-Elliott channel proxy, and the Mininet dumbbell topology.	abr_controller.py, network_channel.py, mininet_topology.py
Soniya Jhadav (PRN: 1262243503)	Benchmarking and Dashboard Lead. Built the 60 s evaluation trace, Pensieve QoE scoring, the 4-panel plotter, the web dashboard and the unit test suite.	benchmark.py, web_dashboard.py, adaptive_streaming_simulator.html, test_simulator.py